

AN ENGINEER'S MENTAL MODEL

Kubernetes

Core concepts, explained one idea at a time

Pods

Deployments

Services

Ingress

Storage

Scaling



A mental model, built one concept at a time



Why orchestration

The problem a fleet of containers creates



Cluster anatomy

Control plane and worker nodes



Workloads

Pods, Deployments, StatefulSets, Jobs



Configuration

ConfigMaps, Secrets, Namespaces



Networking

Services, Ingress, the cluster network



Storage

Volumes, PVs, claims, storage classes



Reliability & access

Probes, autoscaling, resources, RBAC



Talking to it

kubectl and the wider ecosystem

● SECTION 01

Why orchestration exists

One container is easy. A fleet across many machines is a different problem entirely.



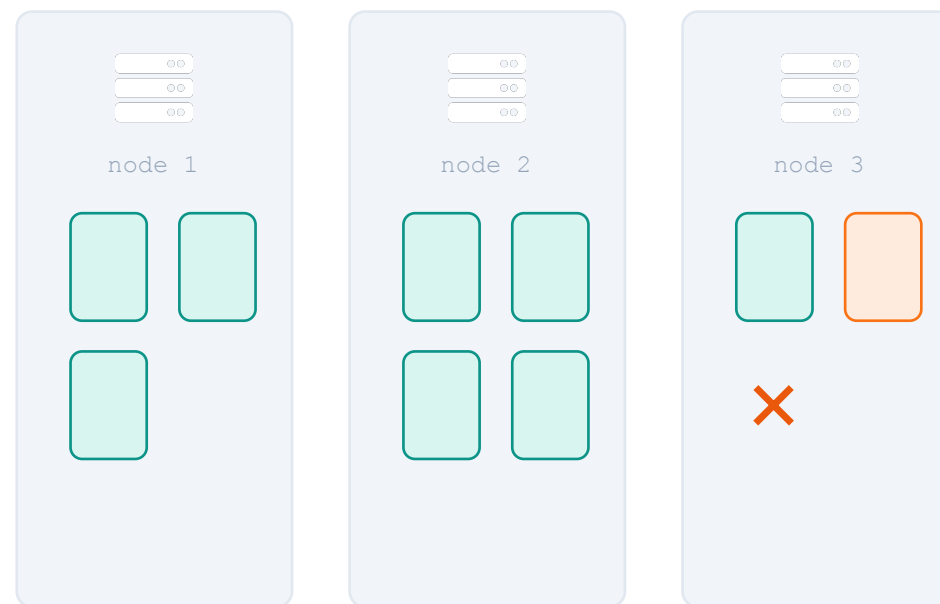
You have 40 containers on 5 machines. Now what?



The questions containers don't answer

- If a container crashes at 3 a.m., who restarts it?
- Which machine has room to run the next one?
- How do 40 moving containers find each other?
- How do you update all of them with zero downtime?
- How do you add copies when traffic spikes — and remove them after?

Doing this by hand does not scale



Kubernetes automates every one of these decisions.

DEFINITION

What Kubernetes actually is



A container orchestrator — a system that runs your containers across a cluster of machines and keeps them running in the state you asked for, without you managing individual hosts.

WHERE IT CAME FROM

Borg

Google's internal cluster manager,
run for a decade



2014

Open-sourced as Kubernetes,
written in Go



CNCF

Donated to the Cloud Native
Computing Foundation



Standard

The de-facto way to run containers in
production

WHAT DEFINES IT

Declarative

Self-healing

Portable

Extensible

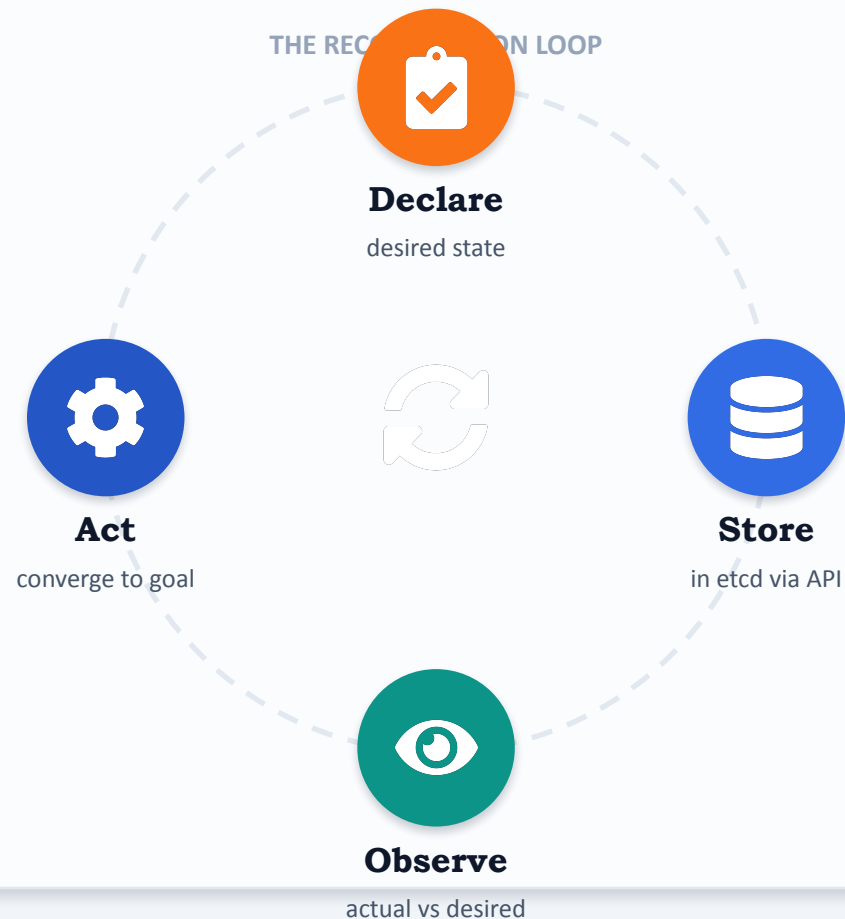
API-driven

Desired state and the reconciliation loop

You describe *what* you want —
not *how* to get there.

- You submit desired state: “run 3 copies of this image.”
- Controllers watch the actual state of the cluster.
- When the two differ, they act to close the gap.
- This never stops — it is a continuous loop.

```
declare  
  
$ kubectl apply -f app.yaml  
# you state the goal; K8s drives
```



● SECTION 02

Cluster anatomy

A cluster is a control plane that makes decisions and worker nodes that run your containers.

The shape of a Kubernetes cluster



CONTROL PLANE · the brain (makes decisions, never runs your app)



API server



etcd



scheduler



controllers

WORKER NODES · where your Pods actually run



node 1

kubelet · kube-proxy · runtime



Pod



Pod



node 2

kubelet · kube-proxy · runtime



Pod



Pod



node 3

kubelet · kube-proxy · runtime



Pod



Pod

The control plane, component by component



kube-apiserver

The front door. Every read and write — from kubectl, controllers, or nodes — goes through this REST API. Nothing talks to the cluster except through it.



etcd

The cluster's single source of truth. A consistent key-value store holding the entire desired and observed state. Lose etcd and you lose the cluster's memory.



kube-scheduler

The placement engine. For each unscheduled Pod it picks the best node, weighing resource requests, affinity rules, taints, and available capacity.



controller-manager

The reconcilers. A bundle of control loops (ReplicaSet, Node, Job...) each watching for drift between desired and actual state and acting to close it.

The worker node, component by component



kubelet

The node's agent. It takes the Pod specs assigned to this node, tells the container runtime to start them, and continuously reports health back to the API server.



kube-proxy

The node's traffic cop. It programs the networking rules that make a Service's single virtual IP route to the right Pods, on any node.



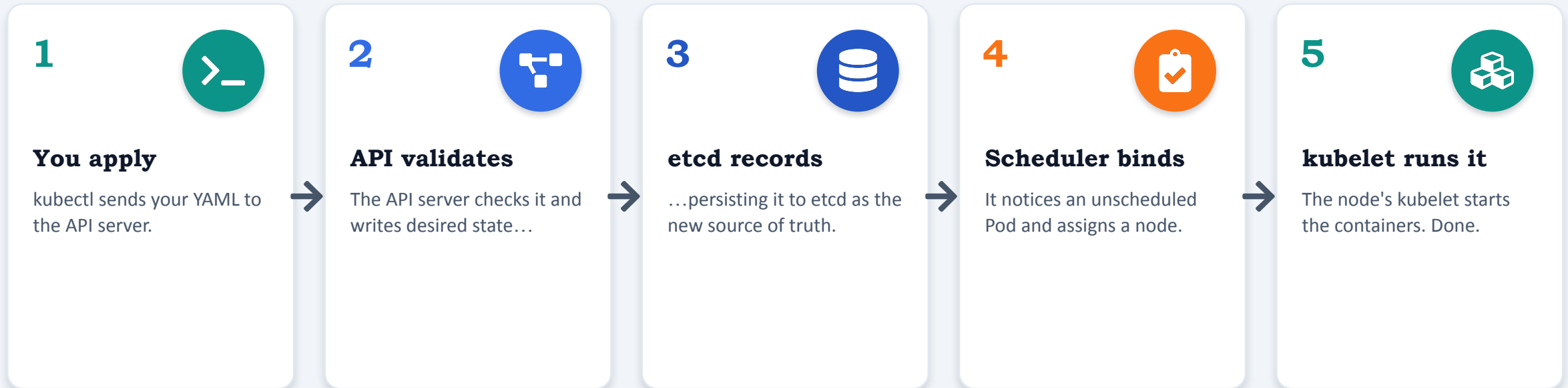
container runtime

The engine that actually pulls images and runs containers — containerd or CRI-O. Kubernetes talks to it through the standard CRI interface.



The takeaway: the control plane decides, the node executes. Your application containers only ever run on worker nodes — never on the control plane.

What happens when you deploy something



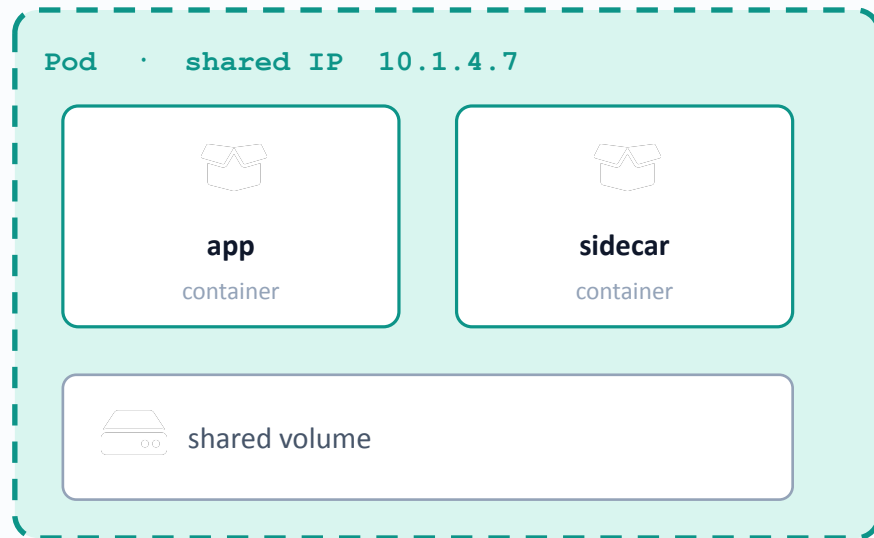
Notice the pattern: no component gives orders directly. Each watches the API for state it cares about and reacts. That loose coupling is what makes Kubernetes resilient.

● SECTION 03

Running workloads

The objects you actually create to run software — from a single Pod up to self-healing, self-updating fleets.

The Pod — the smallest thing you deploy



Containers in a Pod share network and storage.



One shared network

Containers in a Pod reach each other over localhost and share one IP address.



Shared storage

They can mount the same volumes to pass files between them.



Always together

A Pod is scheduled, started, and stopped as one unit — never split across nodes.



Ephemeral by design

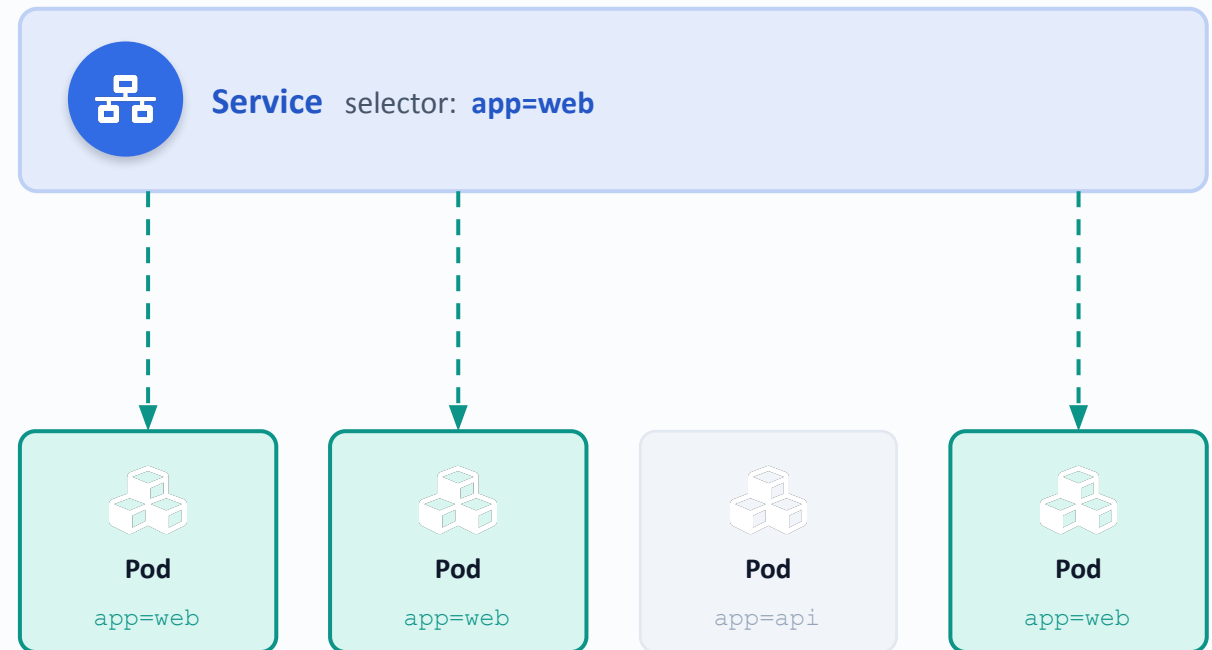
Pods are cattle, not pets. They get replaced, not repaired — with a new IP each time.

Labels & selectors — the glue

Almost nothing in Kubernetes references another object by name.

- Labels are key/value tags you attach to objects.
- A selector is a query: “any Pod where app=web.”
- Services, Deployments and more use selectors to find the Pods they manage.
- Loose coupling: match by attribute, not identity.

```
metadata
labels:
  app: web
  tier: frontend
```



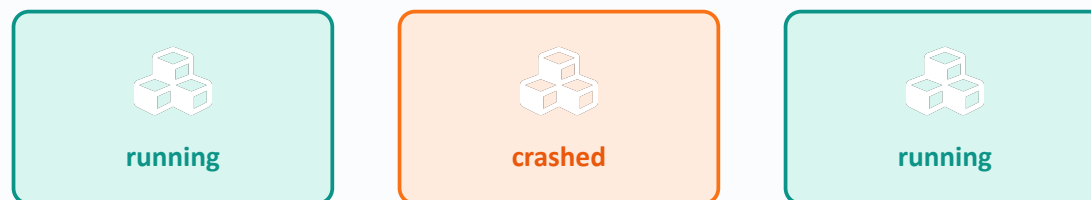
The Service routes only to Pods whose labels match its selector.

ReplicaSet — keep N copies alive

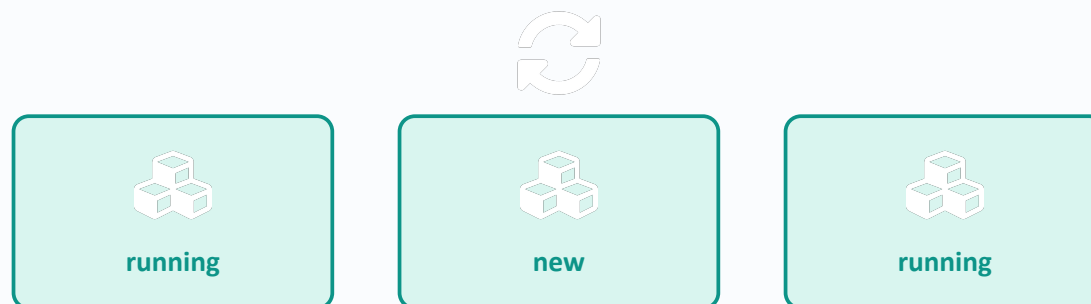
Self-healing, in one object

- You declare a replica count — say, 3.
- The ReplicaSet controller counts running Pods.
- Too few? It creates more. Too many? It deletes some.
- A node dies and takes Pods with it → replacements appear elsewhere, automatically.
- You rarely make these by hand — a Deployment manages them for you.

DESIRED: `replicas = 3`



A Pod dies — actual state (2) drifts below desired (3).



The controller creates a replacement. Back to 3.

Deployment — updates & rollbacks, declaratively

The object you'll use most

- Wraps ReplicaSets and manages them for you.
- Change the image tag and re-apply — it performs a rolling update.
- New Pods scale up as old ones scale down: no downtime.
- Every change is a revision you can roll back to.

```
rollout  
  
$ kubectl rollout status  
deploy/web  
  
$ kubectl rollout undo  
deploy/web
```

ROLLING UPDATE · v1 → v2



At no point are zero Pods serving traffic.

StatefulSet — when identity matters



A Deployment treats its Pods as interchangeable. **A StatefulSet gives each Pod a stable identity and its own storage** — for software that can't be shuffled freely, like databases and queues.



Stable names

Pods are named predictably — db-0, db-1, db-2 — and keep those names across restarts.



Sticky storage

Each Pod keeps its own PersistentVolume. db-0 always reattaches to db-0's data.



Ordered ops

Pods start, update, and terminate in order — essential for clustered, quorum-based apps.

DaemonSet — one Pod on every node

For node-level agents

- Runs exactly one copy of a Pod on each node.
- Add a node to the cluster → it gets the Pod automatically.
- Remove a node → that Pod goes with it.
- Perfect for things every node needs.

TYPICAL USES

log shipper

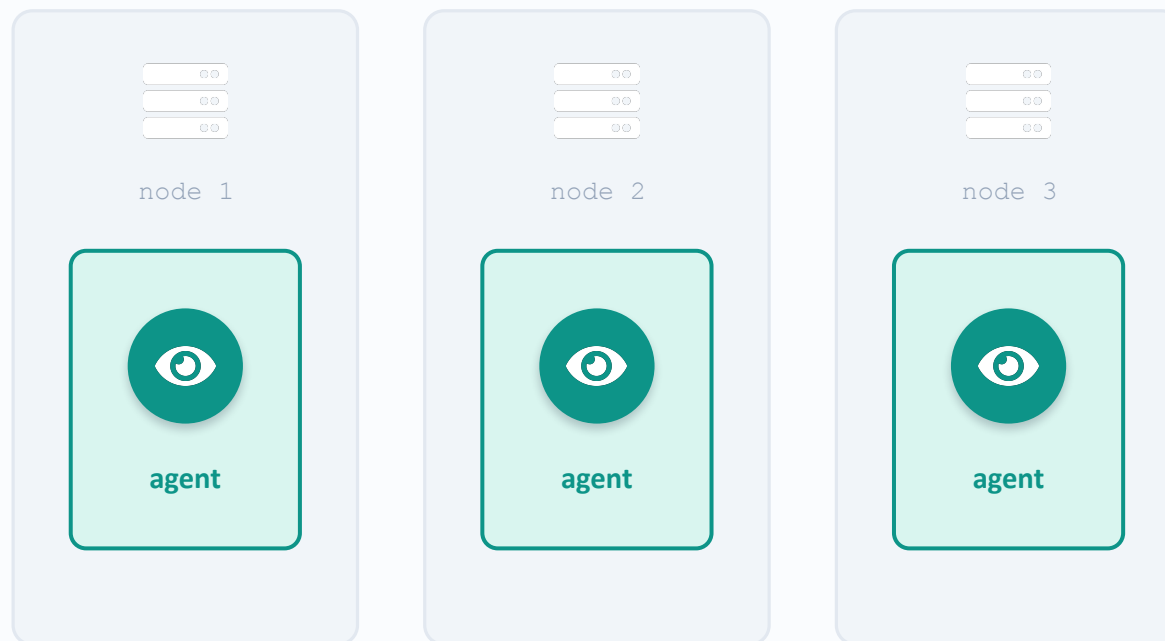
metrics

CNI

storage driver

security agent

Same agent Pod, replicated per node



Job & CronJob — work that finishes



Job

Run to completion, then stop

- Starts one or more Pods and waits for them to succeed.
- Retries on failure up to a limit you set.
- For batch work: a data migration, a report, a backup.



CronJob

A Job on a schedule

- Creates a Job on a cron schedule (e.g. every night).
- Same retry and completion semantics as a Job.
- For recurring work: nightly backups, cleanups, syncs.



Contrast: Deployments, StatefulSets and DaemonSets run forever. Jobs and CronJobs are the objects for work that has a finish line.

● SECTION 04

Configuration

Keep settings and secrets out of your images — and carve one cluster into tidy, separate spaces.



ConfigMaps & Secrets — config out of the image

Same image, every environment. Bake config into the container and you rebuild for every change. Instead, keep it in the cluster and inject it as environment variables or mounted files.



ConfigMap

Non-sensitive key/value settings — feature flags, URLs, tuning values. Change it and re-roll Pods to pick up the new values.



Secret

The same idea for sensitive data — passwords, tokens, TLS keys. Stored separately and mounted only where needed.



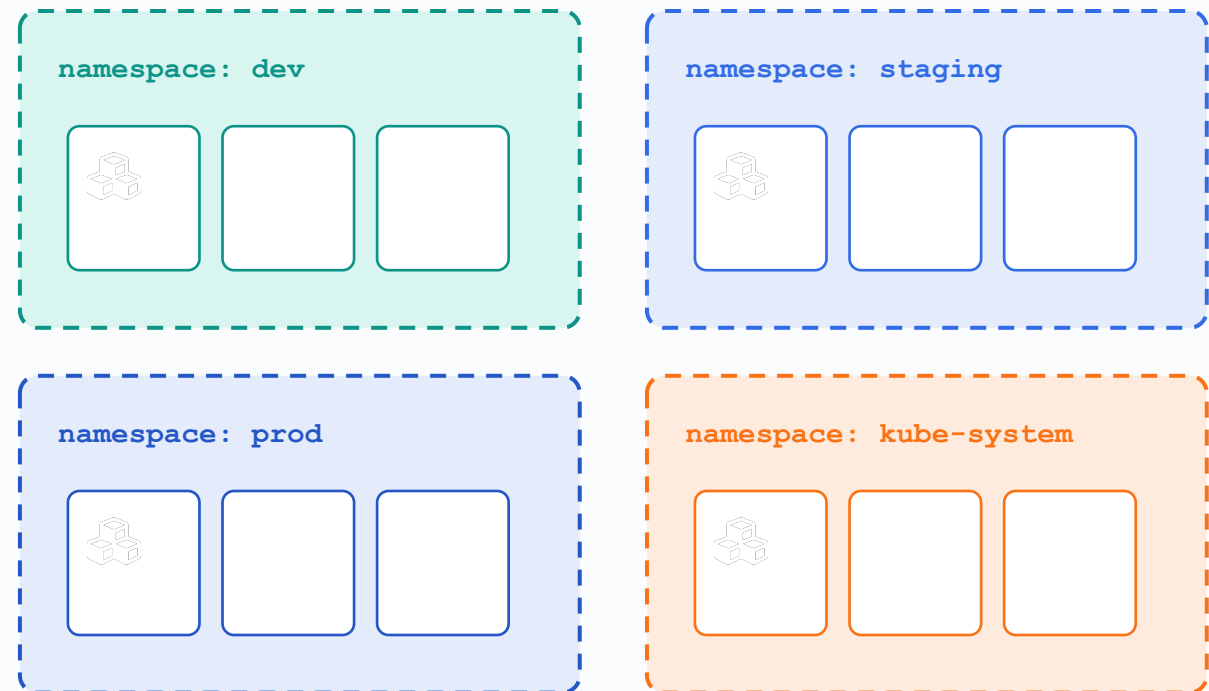
Mind the gap: a Secret is only base64-encoded, not encrypted. Real protection comes from encryption at rest in etcd and tight RBAC on who can read them.

Namespaces — one cluster, many spaces

A virtual cluster inside your cluster

- A scope for names — two teams can both have a “web” Service without clashing.
- A place to attach resource quotas and access rules (RBAC).
- kube-system holds the cluster's own components.
- Not a hard security wall by default — network policy still needed to isolate traffic.

ONE PHYSICAL CLUSTER



● SECTION 05

Networking

How Pods find and reach each other — and how traffic from the outside world gets in.

The Kubernetes network model



Every Pod gets an IP

Not a port on a shared host — its own routable address, like a tiny VM.



Flat, no NAT

Any Pod can reach any other Pod directly by IP, on any node in the cluster.



A plugin does it

The rules are implemented by a CNI plugin — Calico, Cilium, Flannel — not by K8s core.

Because the network is flat, addressing is simple — but Pod IPs still churn constantly:



Pod

10.1.2.5



Pod

10.1.6.9



Pod

10.1.4.2



Pod

10.1.7.8

→ which is exactly why you need a Service.

Service — a stable address for moving Pods

Pods come and go. Their clients shouldn't care.

- A Service is one stable virtual IP and DNS name.
- It selects Pods by label and load-balances across them.
- Pods can die and be replaced — the Service's address never changes.
- Clients connect to the name; K8s handles the rest.



ClusterIP

Reachable only inside the cluster. The default — for service-to-service traffic.



NodePort

Opens the same port on every node's IP. A simple way in from outside, mostly for testing.



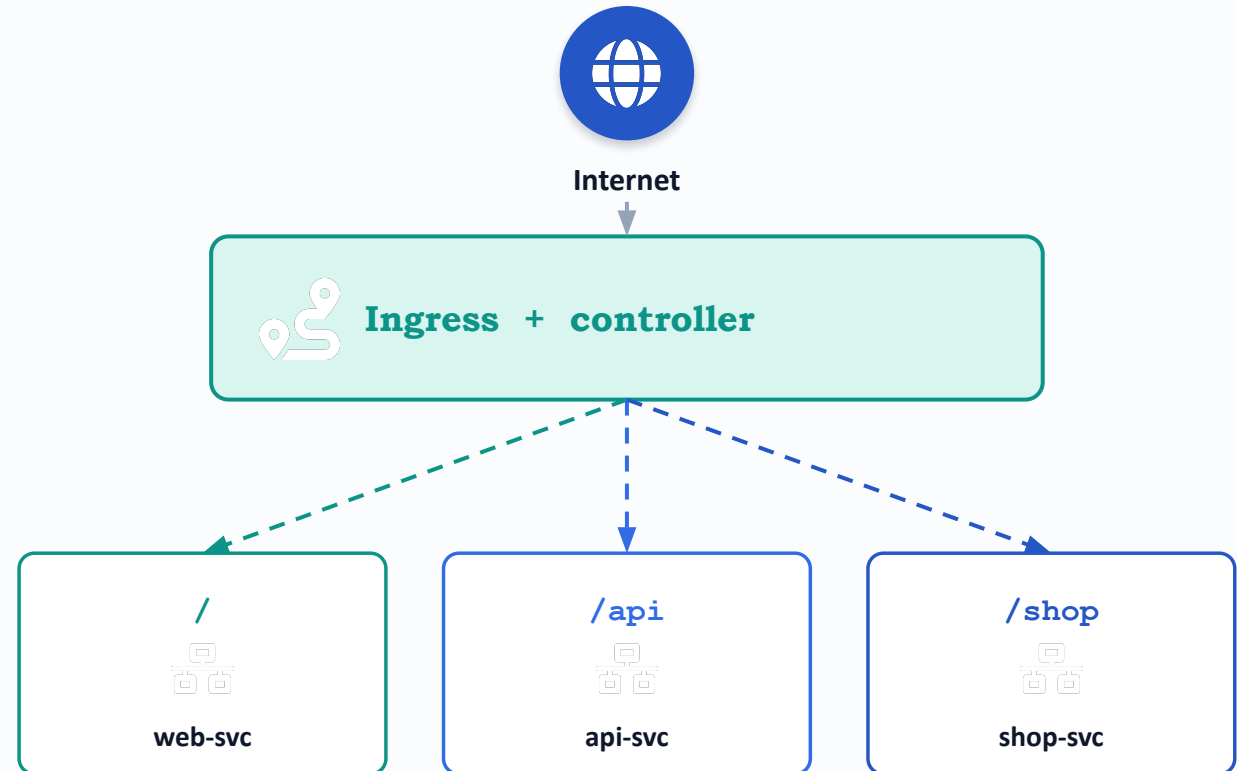
LoadBalancer

Asks the cloud for an external load balancer with a public IP. The production front door.

Ingress — smart HTTP routing at the edge

One entry point, many backends

- A LoadBalancer per Service gets expensive fast.
- Ingress is one HTTP(S) entry that routes by host and path to different Services.
- It terminates TLS in one place.
- It only defines rules — an ingress controller (nginx, Traefik) actually enforces them.



● SECTION 06

Storage

Containers are ephemeral. Storage abstractions let data outlive the Pod that wrote it.

Volumes, PVs, claims & storage classes

The core problem: a container's filesystem dies with it. Kubernetes separates “what a Pod asks for” from “what storage actually exists,” so data can persist and Pods stay portable.



Volume

A directory available to the Pod's containers. An emptyDir dies with the Pod; a claim-backed volume persists.



PersistentVolumeClaim

A Pod's request for storage: “I need 10Gi, read-write.” It doesn't care where it comes from.



PersistentVolume

An actual chunk of storage in the cluster — a cloud disk, NFS share, etc. Bound to a claim.



StorageClass

A template that creates PVs automatically when a claim appears — no manual disk wrangling.

● SECTION 07

Keeping workloads healthy

Give the scheduler what it needs, let Kubernetes detect failure, and scale to meet load.

Resource requests & limits



requests — the guarantee

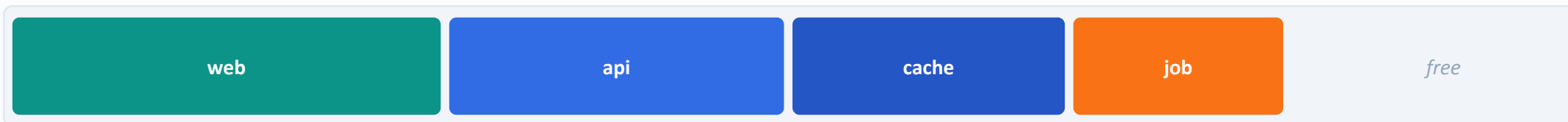
What a container is guaranteed. The scheduler uses requests to decide which node has room — it's the basis for placement and bin-packing.



limits — the ceiling

The hard cap. Exceed the CPU limit and you're throttled; exceed the memory limit and the container is OOM-killed and restarted.

Requests let the scheduler pack nodes without over-committing:



Health probes — how K8s knows a Pod is OK



Liveness

“Are you still alive?”

On failure: the container is restarted. Catches deadlocks and hung processes.



Readiness

“Ready for traffic?”

On failure: the Pod is pulled out of its Service — no restart. Traffic resumes when it passes again.



Startup

“Finished booting?”

Holds the other probes off until a slow app has started, so it isn't killed mid-boot.

The difference that trips people up: liveness restarts a broken Pod; readiness just stops sending it traffic until it recovers.

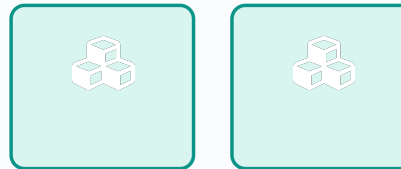
Horizontal Pod Autoscaler

Match capacity to load, automatically

- Watches a metric — CPU, memory, or a custom one.
- Adjusts the replica count between a min and max you set.
- Load rises → more Pods. Load falls → fewer Pods.
- Needs metrics-server installed to read usage.

Not the same as the **Cluster Autoscaler**, which adds and removes whole nodes.

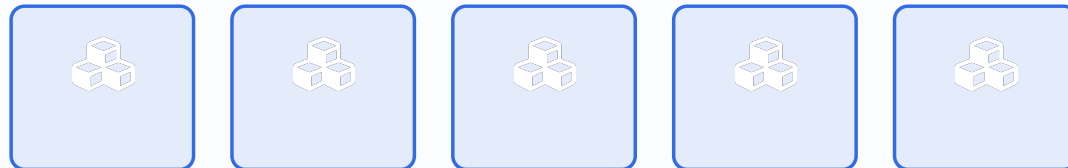
CPU 30% — steady



load spikes



CPU 85% — scale out



RBAC — who is allowed to do what

Every request to the API is authorized. Role-Based Access Control answers “can this identity perform this verb on this resource?” — built from three simple pieces.



Subject

Who is acting — a human user or, for in-cluster workloads, a ServiceAccount attached to a Pod.



Role

What's allowed — a set of verbs (get, list, create...) on resources, scoped to a namespace or the whole cluster.



RoleBinding

The link — grants a Role to a Subject. No binding, no access.

Guiding principle: least privilege — grant the narrowest set of verbs and resources a subject actually needs.

● SECTION 08

Talking to the cluster

One client, one API — plus the ecosystem that turns Kubernetes into a full platform.

kubectl — your line to the API server

Everything is an API object

- kubectl is a thin client — it just talks to the API server.
- Declarative: apply a YAML file and let controllers converge.
- Imperative verbs are handy for inspecting and debugging.
- The same objects, whether you use YAML, a UI, or CI.

get

describe

logs

apply

exec

rollout

```
kubectl

$ kubectl apply -f deploy.yaml
deployment.apps/web configured

$ kubectl get pods -l app=web
NAME          READY   STATUS
web-7d..-x    1/1     Running
web-7d..-q    1/1     Running

$ kubectl logs -f web-7d..-x
listening on :8080
```

Kubernetes is a platform for building platforms



Helm

Package and template whole apps as versioned, installable charts.



Operators

Custom controllers that encode operational know-how for a specific app.



GitOps

Git is the source of truth; a tool (ArgoCD, Flux) reconciles the cluster to it.



Observability

Prometheus, Grafana and friends for metrics, logs, and traces.



Service mesh

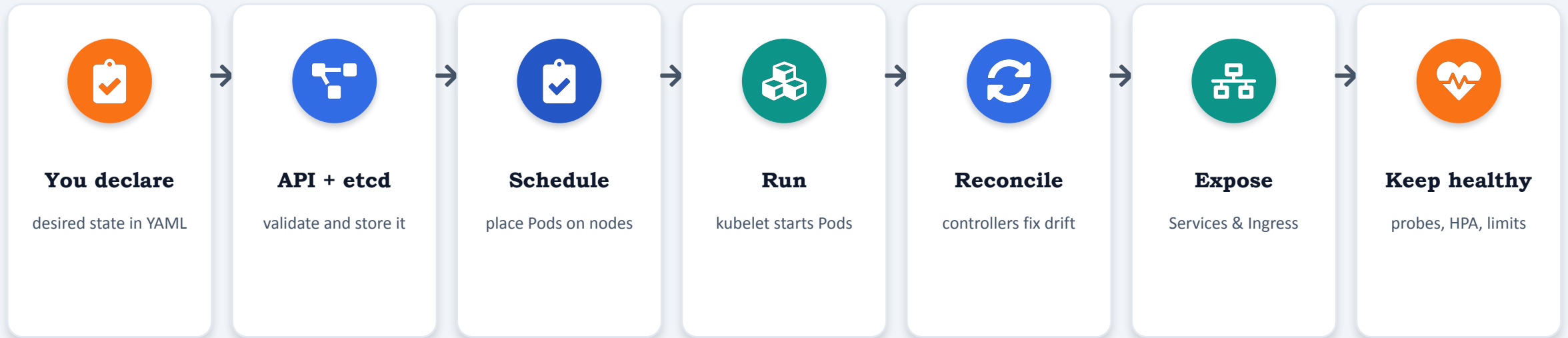
Istio, Linkerd add traffic control, mTLS, and telemetry between services.



CRDs

Extend the API with your own object types — the cluster becomes programmable.

The whole model in one picture



And it never stops. The loop runs forever — you change the desired state, Kubernetes quietly drives the cluster back to matching it. Master that one idea and the rest is detail.

THE ONE-LINE TAKEAWAY

You describe the destination.
Kubernetes drives.



Declarative

Self-healing

Everything is an object

Loosely coupled